

09/12

REST APIs

(Representational State Transfer)

Initially the internet was created by Tim Berners-Lee, as a method to share scientific knowledge easy & universal. So in the 1990s, he made a server and browser building.

① HTML :

② HTTP :

③ URL :

Then later CERN made the software royalty free.

④ Where does REST come in ?

→ As the web grew fast, scaling and integrating large field became a challenge.

→ Roy fielding, suggested REST.

⑤ What is REST ?

REST defined the principles for designing scalable, stateless and uniform APIs - using HTTP not just for pages, but for everything in modern backend engineering.

- ① Statelessness: Each req carries all necessary context, server doesn't retain per-user state
- ② Cacheability: REQ/RES must indicate if they can be cached to improve speed & scalability.
- ③ Resource-Based: REST treats everything like resource - each identified by unique URL
- ④ Standard Actions: (1) GET (2) POST (3) PUT/PATCH (4) DELETE
- ⑤ Uniform Interface: The client & server talk in a consistent, predictable way: a fixed set of operations, known as message format (usually JSON)
- ⑥ Separation of concerns: The client & server are completely independent, each can

change without breaking the other.

Any service or API following these principles are called RESTful

09/15

REST URLs

The idea is to provide a simple, unique address (URL) for every resource, which makes it easy for clients to find, retrieve & interact with data in a standardized way.

Key points

1) Resource Identification

→ Every piece of data (book, resource) is called a resource

→ Each resource is identified by its own absolute URL, so the client can act on that data.

2) URL structure

- 1) use nouns, not verb,
- 2) Collections use plural nouns- → hierarchy
- 3) Single resource : /users/42 , /books/43
- 4) Nested resource

3) Parameters & Filtering

(-) Query Params add search, sort or filter

4) Hierarchical & Predictable

(-) URLs are hierarchical, readable & Predictable.

II Idempotency

Idempotency means that in HTTP & REST means making the same request multiple times resulting the same effect as making it once.

HTTP Method	Idempotent	Example
GET	Yes	1000 times Fetching data
PUT	Yes	Replaces a resource
DELETE	Yes	Deletes a resource
POST	No	Each repetition

PATCH	No*	Not
-------	-----	-----

(6) Why does Idempotency matter?

→ Makes retries safer

→ Reduces surprises : Makes system predictable

→ Aligned with HTTP spec: Many clients assume certain methods are idempotent.

If we don't follow REST principles, we end up making our API confusing, tightly coupled, hard to maintain & overall a poor user experience.

Design Process

1] Identify the resources

(i) list out logical entities

(ii) Think from client's needs

2] Resource Modelling :

(a) Group related data into collections.

(b) Use singular resource for single entities

(c) Decide on granularity:- combine related data into logical units

3] Design URLs & Methods

(i) Use clean, noun-based URLs

(i) GET /orders/23

(i) POST /books

(i) PUT /users/53

4] Wireframe & Data Contracts

(i) Draft example requests & responses for every resource

(i) Wireframe the flows

(i) Keep things predictable & consistent

5] Database Schema Design

(i) The API resource doesn't have to mirror DB schema directly.

(i) Backing tables or views should efficiently provide Resource data.

09/16

Pagination

Listing all items at once, doesn't scale
- too much data for server, network and
user. Users usually want something,
fast, manageable subset of the data at a time.

Goal: Deliver results in actionable
chunks, each API call or page view
gives a "slice" of data.